



Sistema de Gestión de Torneos de Tiro con Arco

Mongo Edition

Grupo 1

Integrantes:

- Martín Alvayay
- Martin Araya
- José Ceardi
- Benjamín Letelier
- Benjamín Paredes
- Nicolas Rojas

Distribución

del

Equipo

3 Bases de datos

Martín Alvayay
Martín Araya
Benjamín Paredes

2 Backend

José Ceardi
Nicolás Rojas

3 Frontend

Martín Alvayay
José Ceardi
Benjamín Letelier

El Desafío: Organizar Torneos de Tiro con Arco

Gestionar torneos de tiro con arco implica coordinar inscripciones, puntuaciones, clasificaciones y comunicaciones de forma simultánea. Sin una herramienta adecuada, los errores manuales y la desorganización afectan tanto a los organizadores como a los competidores.

Para los Organizadores

Reducir la carga administrativa y minimizar errores en la gestión del evento.

Para los Arqueros

Acceso ágil a inscripciones, resultados y clasificaciones en tiempo real.

https://www.worldarchery.sport/rhttps://www.worldarchery.sport/rulebook/article/793?book_title=Cap%C3%ADtulo%2011%20-%20Equipo%20de%20los%20Atletas

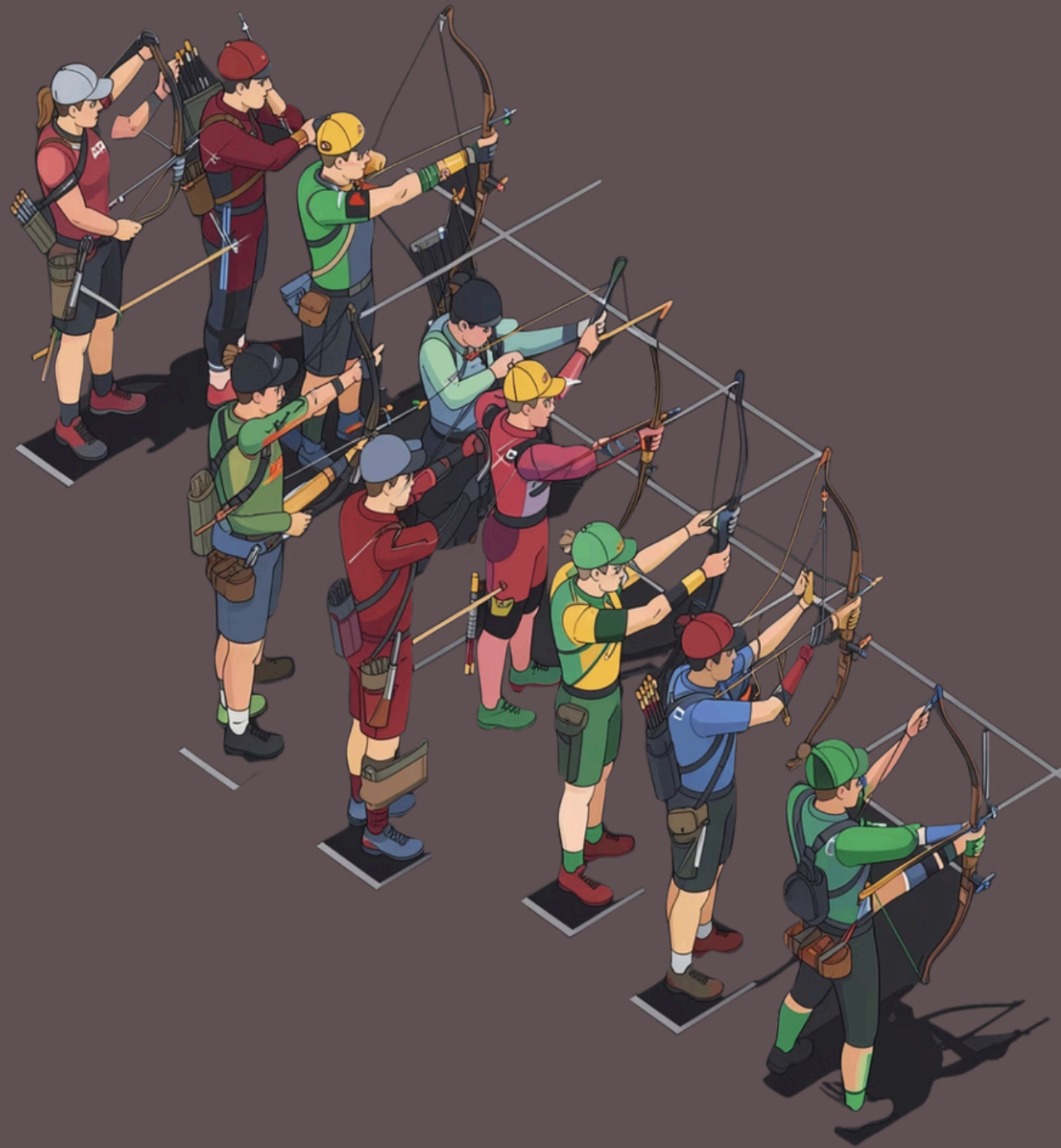
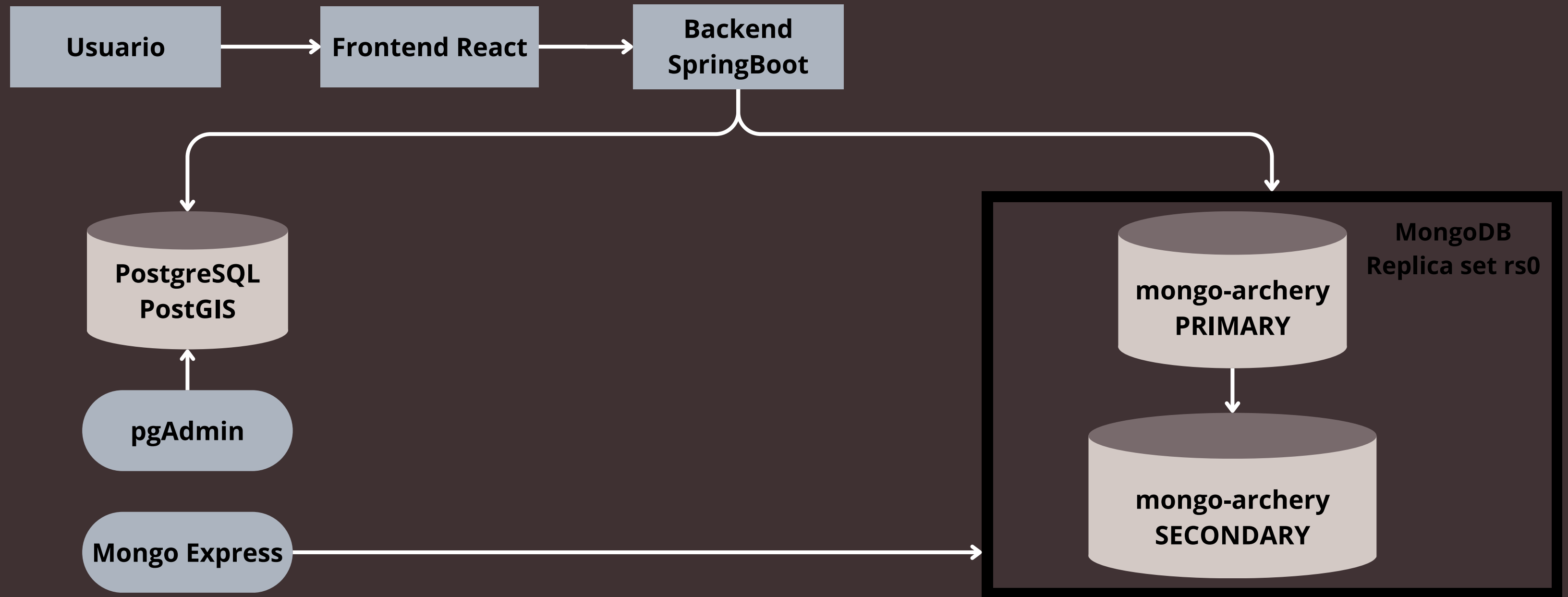


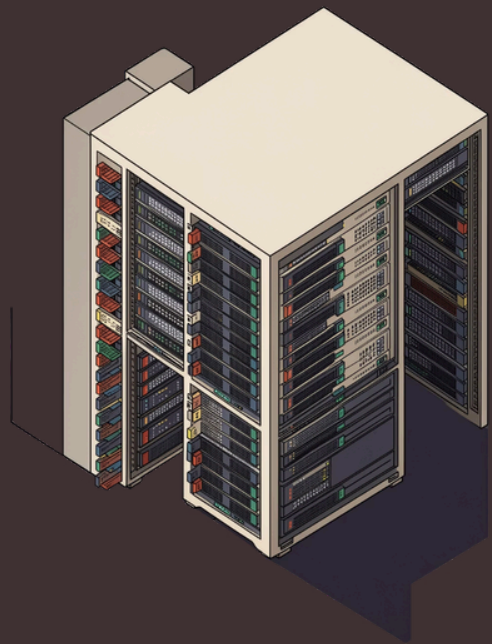
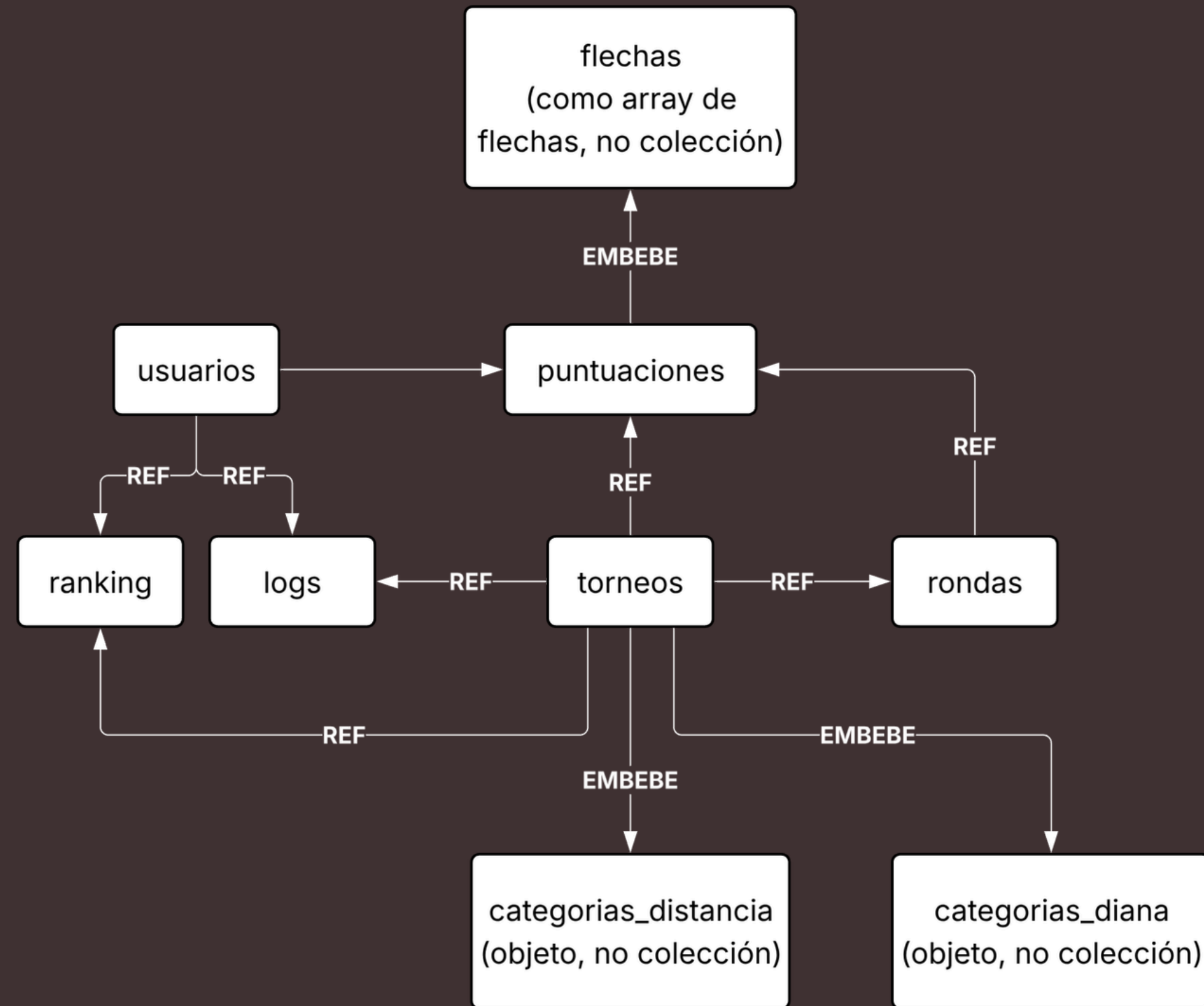
Diagrama de la arquitectura

Arquitectura por capas

Diseñamos un sistema integral que cubre todo el ciclo del torneo, estructurado sobre tres pilares tecnológicos fundamentales



Modelado de Datos Orientado a Documentos



Demostración de Funcionalidades



Modelado de Datos:

Las puntuaciones de cada ronda están referenciadas en una colección independiente.

```
public class TorneoDocument {

    @Id
    private String id;

    @Indexed(unique = true)
    private String nombre;

    private String estado = "PENDIENTE";
    private LocalDate fechaInicio;
    private LocalDate fechaTermino;
    private int plazasMax;
    private int plazasActual = 0;

    private Long categoriaDistanciaId;
    private CategoriaDistanciaEmbedded categoriaDistancia;

    private Long categoriaDianaId;
    private CategoriaDianaEmbedded categoriaDiana;

    private String zonaCompetenciaGeoJSON;
    private String lineaTiroGeoJSON;

    @Data 3 usages Joseardi
    @NoArgsConstructor
    @AllArgsConstructor
    public static class CategoriaDistanciaEmbedded {
        private String nombre = "RECURVO";
        private int distanciaTiro = 18;
    }

    @Data 3 usages Joseardi
    @NoArgsConstructor
    @AllArgsConstructor
    public static class CategoriaDianaEmbedded {
        private String nombre = "ADULTO";
        private int puntajeMinimo = 5;
    }
}
```

```
public PuntuacionDocument guardarOActualizar(PuntuacionDocument doc) {
    validarDatosEntrada(doc);
    Usuario usuario = validarUsuario(doc.getUsuarioId());
    TorneoDocument torneo = validarTorneoEnCurso(doc.getTorneoId());
    validarRondaEnCurso(doc.getRondaId(), doc.getTorneoId());
    validarInscripcion(doc.getTorneoId(), doc.getUsuarioId());
    return ejecutarUpsert(doc, usuario, torneo);
}
```

Transacción Multi-Documento:

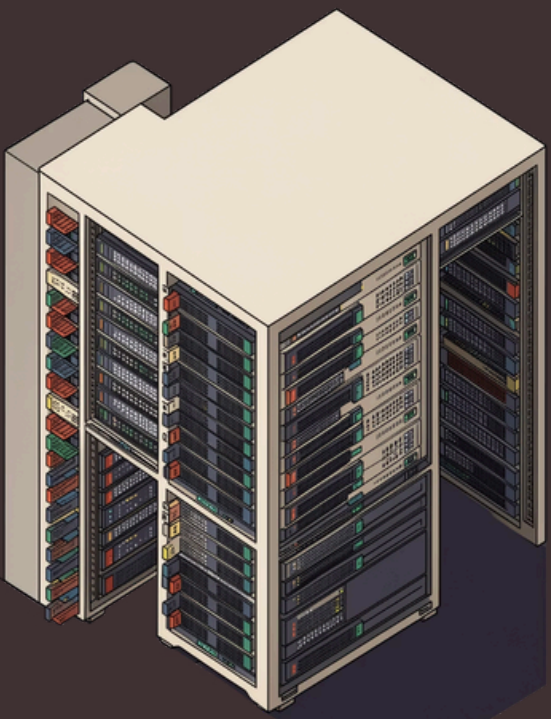
Transacción que actualice de forma atómica el documento de la ronda, el acumulado del arquero y el ranking del torneo.

```
@Transactional
public PuntuacionDocument guardarOActualizarConReglamento(PuntuacionDocument doc) {
    validarDatosEntrada(doc);
    Usuario usuario = validarUsuario(doc.getUsuarioId());
    TorneoDocument torneo = validarTorneoEnCurso(doc.getTorneoId());
    validarRondaEnCurso(doc.getRondaId(), doc.getTorneoId());
    validarInscripcion(doc.getTorneoId(), doc.getUsuarioId());
    validarReglamentoCategoria(doc, torneo);
    validarPosicionArquero(doc, torneo);
    validarDistanciaTiro(doc, torneo);

    PuntuacionDocument saved = ejecutarUpsert(doc, usuario, torneo);
    actualizarParticipacion(doc.getTorneoId(), doc.getUsuarioId());
    actualizarRankingVivo(doc.getTorneoId(), usuario.getIdUsuario(), usuario.getNombre(), torneo.getNombre(), saved.getPuntajeTotal());
    return saved;
}
```


Validación de Esquema:

```
applyValidator("puntuaciones", {
  bsonType: "object",
  required: ["torneoId", "rondaId", "usuarioId", "puntajeMinimo", "flechas", "puntajeTotal"],
  properties: {
    torneoId: { bsonType: "string", minLength: 1 },
    rondaId: { bsonType: "string", minLength: 1 },
    usuarioId: { bsonType: "long" },
    nombreArquero: { bsonType: "string" },
    nombreTorneo: { bsonType: "string" },
    numeroRonda: { bsonType: "int" },
    categoria: { bsonType: "string" },
    puntajeMinimo: { bsonType: "int", minimum: 0 },
    flechas: {
      bsonType: "array",
      minItems: 1,
      maxItems: 6,
      items: { bsonType: "int" }
    },
    puntajeTotal: { bsonType: "int", minimum: 0, maximum: 60 },
    posicionArquero: { bsonType: "string" },
    posicionDiana: { bsonType: "string" },
    createdAt: { bsonType: "date" },
    updatedAt: { bsonType: "date" }
  }
})
```



Aggregation Pipeline:

Pipeline que calcula el promedio de puntaje y la mejor ronda por arquero, agrupado por categoría y torneo.

```
3 const pipeline : [{$project: {torneoId: number,...} = [
4   { $project: { torneoId: 1, nombreTorneo: 1, categoria: 1, usuarioId: 1, nombreArquero: 1, numeroRonda: 1, puntajeTotal: 1 } },
5   {
6     $facet: {
7       rendimientoPorTorneo: [
8         { $group: {
9           _id: { torneoId: "$torneoId", usuarioId: "$usuarioId" },
10          nombreTorneo: { $first: "$nombreTorneo" },
11          nombreArquero: { $first: "$nombreArquero" },
12          promedioPuntaje: { $avg: "$puntajeTotal" },
13          mejorRonda: { $top: { sortBy: { puntajeTotal: -1, numeroRonda: 1 }, output: { numeroRonda: "$numeroRonda", puntaje: "$puntajeTotal" } } },
14          rondasRegistradas: { $sum: 1 }
15        } },
16        { $project: { _id: 0, torneoId: "$_id.torneoId", usuarioId: "$_id.usuarioId", nombreTorneo: 1, nombreArquero: 1, promedioPuntaje: 1, mejorRonda: 1, rondasRegistradas: 1 } },
17        { $sort: { torneoId: 1, promedioPuntaje: -1, "mejorRonda.puntaje": -1, usuarioId: 1 } }
18      ],
19      rendimientoPorCategoria: [
20        { $group: {
21          _id: { categoria: "$categoria", usuarioId: "$usuarioId" },
22          nombreArquero: { $first: "$nombreArquero" },
23          promedioPuntaje: { $avg: "$puntajeTotal" },
24          mejorRonda: { $top: { sortBy: { puntajeTotal: -1, numeroRonda: 1 }, output: { numeroRonda: "$numeroRonda", puntaje: "$puntajeTotal" } } },
25          rondasConsideradas: { $sum: 1 }
26        } },
27        { $project: { _id: 0, categoria: "$_id.categoria", usuarioId: "$_id.usuarioId", nombreArquero: 1, promedioPuntaje: 1, mejorRonda: 1, rondasConsideradas: 1 } },
28        { $sort: { categoria: 1, promedioPuntaje: -1, "mejorRonda.puntaje": -1, usuarioId: 1 } }
29      ],
30      distribucionPorRendimiento: [
31        { $group: { _id: { categoria: "$categoria", usuarioId: "$usuarioId" }, nombreArquero: { $first: "$nombreArquero" }, promedioPuntaje: { $avg: "$puntajeTotal" }, rondasConsideradas: { $sum: 1 } } },
32        { $bucket: {
33          groupBy: "$promedioPuntaje",
34          boundaries: [0, 15, 30, 45, 61],
35          default: "fuera_de_rango",
36          output: {
37            cantidadDesempenos: { $sum: 1 },
38            arqueros: { $push: { categoria: "$_id.categoria", usuarioId: "$_id.usuarioId", nombreArquero: "$nombreArquero", promedioPuntaje: "$promedioPuntaje", rondasConsideradas: "$rondasConsideradas" } }
39          }
40        } }
41      ],
42    }
43  }
44  ];
```

Índices:

```
const indexName = puntuaciones.createIndex(  
  { torneoId: 1, rondaId: 1, usuarioId: 1 },  
  {  
    name: "uk_puntuacion_torneo_ronda_usuario",  
    unique: true  
  }  
);
```

```
const sample = puntuaciones.findOne(  
  {},  
  { _id: 0, torneoId: 1, rondaId: 1, usuarioId: 1 }  
);  
  
const scoringFilter = sample || {  
  torneoId: "torneo-de-ejemplo",  
  rondaId: "ronda-de-ejemplo",  
  usuarioId: NumberLong("0")  
};
```

```
printjson(  
  puntuaciones  
    .find(scoringFilter)  
    .explain("executionStats")  
);
```

Índice compuesto único que impide el doble registro de una puntuación en la misma ronda, y evalúa con explain() su impacto en el rendimiento.

Change Streams:

Listener que, ante cada inserción, actualice en tiempo real una colección materializada de “ranking en vivo” del torneo.

```
@PostConstruct
public void registrarListener() {
    ChangeStreamRequest.ChangeStreamRequestBuilder<Document> builder = ChangeStreamRequest.builder();
    ChangeStreamRequest<Document> request = builder
        .collection("puntuaciones")
        .filter(Aggregation.newAggregation(
            Aggregation.match(Criteria.where("operationType").is("insert"))
        ))
        .publishTo((MessageListener<ChangeStreamDocument<Document>, Document>) message -> {
            Document document = message.getBody();
            if (document == null) {
                return;
            }
            String usuarioId = document.getString("usuarioId");
            if (usuarioId != null) {
                ejecutarMergeRanking(usuarioId);
            }
        })
        .build();
    listenerContainer.register(request, Document.class);
}
```


Change Streams:

Listener que, ante cada inserción, actualice en tiempo real una colección materializada de “ranking en vivo” del torneo.

```
private void ejecutarMergeRanking(String usuarioId) {
    Aggregation aggregation = Aggregation.newAggregation(
        Aggregation.match(Criteria.where("usuarioId").is(usuarioId)),
        Aggregation.group("usuarioId")
            .sum("puntos").as("puntajeTotal"),
        Aggregation.project("puntajeTotal")
            .and("_id").as("usuarioId")
            .andExclude("_id"),
        MergeOperation.builder()
            .intoCollection("ranking_en_vivo")
            .on("usuarioId")
            .whenMatched(MergeOperation.WhenDocumentsMatch.mergeDocuments())
            .whenNotMatched(MergeOperation.WhenDocumentsDontMatch.insertNewDocument())
            .build()
    );
    mongoTemplate.aggregate(aggregation, "puntuaciones", Document.class);
}
```

Pipeline

```
{
  $facet: {
    rendimientoPorTorneo: [
      { $group: {
        _id: { torneoId: "$torneoId", usuarioId: "$usuarioId" },
        nombreTorneo: { $first: "$nombreTorneo" },
        nombreArquero: { $first: "$nombreArquero" },
        promedioPuntaje: { $avg: "$puntajeTotal" },
        mejorRonda: { $top: { sortBy: { puntajeTotal: -1, numeroRonda: 1 }, output: { numeroRonda: "$numeroRonda", puntajeTotal: "$puntajeTotal" } } },
        rondasRegistradas: { $sum: 1 }
      } },
      { $project: { _id: 0, torneoId: "$_id.torneoId", usuarioId: "$_id.usuarioId", nombreTorneo: 1, nombreArquero: 1, promedioPuntaje: 1, mejorRonda: 1, rondasRegistradas: 1 } },
      { $sort: { torneoId: 1, promedioPuntaje: -1, "mejorRonda.puntaje": -1, usuarioId: 1 } }
    ],
    rendimientoPorCategoria: [
      { $group: {
        _id: { categoria: "$categoria", usuarioId: "$usuarioId" },
        nombreArquero: { $first: "$nombreArquero" },
        promedioPuntaje: { $avg: "$puntajeTotal" },
        mejorRonda: { $top: { sortBy: { puntajeTotal: -1, numeroRonda: 1 }, output: { numeroRonda: "$numeroRonda", puntajeTotal: "$puntajeTotal" } } },
        rondasConsideradas: { $sum: 1 }
      } },
      { $project: { _id: 0, categoria: "$_id.categoria", usuarioId: "$_id.usuarioId", nombreArquero: 1, promedioPuntaje: 1, mejorRonda: 1, rondasConsideradas: 1 } },
      { $sort: { categoria: 1, promedioPuntaje: -1, "mejorRonda.puntaje": -1, usuarioId: 1 } }
    ]
  }
}
```

Vista para salida del Pipeline

```
],
"distribucionPorRendimiento": [
  {
    "_id": 15,
    "cantidadDesempenos": 1,
    "arqueros": [
      {
        "categoria": "COMPUESTO",
        "usuarioId": 101,
        "nombreArquero": "Ana Pérez",
        "promedioPuntaje": 15,
        "rondasConsideradas": 2
      }
    ]
  },
  {
    "_id": 45,
    "cantidadDesempenos": 2,
    "arqueros": [
      {
        ...
      }
    ]
  }
],
"rendimientoPorTorneo": [
  {
    "torneoId": "T1",
    "usuarioId": 101,
    "nombreTorneo": "Torneo Apertura",
    "nombreArquero": "Ana Pérez",
    "promedioPuntaje": 45,
    "mejorRonda": { "numeroRonda": 2, "puntaje": 60},
    "rondasRegistradas": 2
  },
  {
    ...
  }
],
"rendimientoPorCategoria": [
  {
    "categoria": "RECURVO",
    "usuarioId": 101,
    "nombreArquero": "Ana Pérez",
    "promedioPuntaje": 45,
    "mejorRonda": { "numeroRonda": 2, "puntaje": 60},
    "rondasConsideradas": 4
  },
  {
    ...
  }
]
```



Distribución por Rendimiento		
15-29 pts	30-44 pts	45-60 pts
Karla Fuentes Recurvo Indoor · promedio 26.33 · 3 rondas	Javier Contreras Recurvo Escuela · promedio 32.33 · 3 rondas	Camila Rojas Recurvo Escuela · promedio 50.67 · 3 rondas
Lucas Navarro Recurvo Escuela · promedio 27.67 · 3 rondas	Hugo Castillo Recurvo Indoor · promedio 39.00 · 3 rondas	Diego Muñoz Recurvo Escuela · promedio 51.00 · 3 rondas
Karla Fuentes Recurvo Escuela · promedio 27.00 · 3 rondas	Felipe Vargas Recurvo Escuela · promedio 44.33 · 3 rondas	Elena Soto Compuesto · promedio 45.00 · 3 rondas
Lucas Navarro Recurvo Indoor · promedio 27.00 · 3 rondas	Isidora Morales Recurvo Escuela · promedio 33.00 · 3 rondas	Elena Soto Recurvo Indoor · promedio 45.67 · 3 rondas
	Hugo Castillo Recurvo Escuela · promedio 39.67 · 3 rondas	Bruno Silva Recurvo Escuela · promedio 54.67 · 3 rondas
	Gabriela Pérez Recurvo Indoor · promedio 38.33 · 3 rondas	Camila Rojas Compuesto · promedio 50.33 · 3 rondas
	Javier Contreras Recurvo Indoor · promedio 33.00 · 3 rondas	Bruno Silva Recurvo Indoor · promedio 54.67 · 3 rondas

Rendimiento por Torneo

Torneo	Arquero	Promedio	Mejor ronda	Rondas
Copa Santiago Histórica	Ana Torres	54.67	Ronda 1 (59 pts)	3
Copa Santiago Histórica	Bruno Silva	54.67	Ronda 1 (59 pts)	3

Rendimiento por Categoría

Categoría	Arquero	Promedio	Mejor ronda	Rondas
Compuesto	Ana Torres	55.33	Ronda 1 (58 pts)	3
Compuesto	Bruno Silva	55.33	Ronda 1 (58 pts)	3
Compuesto	Camila Rojas	50.33	Ronda 1 (57 pts)	3
Compuesto	Diego Muñoz	50.00	Ronda 1 (56 pts)	2

CONCLUSIONES

Problemas

- Cómo comunicar (NOSQL + SQL + POSTGIS)
- Pruebas antes de commit

Resultados

- Flexibilidad de modelo de datos
- potencia del pipeline
- rendimiento en consultas
- escalabilidad horizontal
- consistencia y operaciones atómicas

Soluciones / aprendizajes

- Mongo y replica set para @Transaccional
- Backend como gestor principal
- Migración en lugar de reconstrucción